

INTERVIEW PREPARATION GUIDE

Telco Customer Churn Prediction

XGBoost · SHAP · FastAPI · Streamlit · MLflow · scikit-learn

Model	CV AUC	Test AUC	PR AUC	F1
Logistic Regression	0.836	0.841	0.631	0.597
Random Forest	0.861	0.858	0.672	0.618
Gradient Boosting	0.867	0.863	0.681	0.625
LightGBM	0.873	0.869	0.693	0.631
XGBoost (final)	0.878	0.874	0.702	0.643

1. Project Overview

This project builds an end-to-end machine learning system to predict customer churn for a telecom company using the IBM Telco Customer Churn dataset. Churn — when a customer stops using a service — is one of the most important business problems in subscription-based industries. The goal is not just to predict who will churn, but to enable the business to act on those predictions through a risk-tiered intervention strategy.

Business Problem

Telecom companies lose 15-25% of customers annually to churn. Acquiring a new customer costs 5-7 times more than retaining an existing one. A churn prediction model allows the retention team to identify at-risk customers early and intervene with targeted offers, reducing overall churn rate without wasting budget on customers who were never going to leave.

Dataset

- Source: IBM Telco Customer Churn dataset (publicly available on Kaggle)
- Size: 7,043 customer records, 20 features, 1 binary target (Churn: Yes/No)
- Churn rate: 26.5% — a moderately imbalanced dataset
- Features: Demographics, services subscribed, contract type, billing, tenure

■ *Interview tip: Start your answer with the business problem before the technical solution. Say: 'The business goal was to reduce customer churn by identifying at-risk customers early, so the retention team could intervene before they left.'*

2. Exploratory Data Analysis (EDA)

EDA is the first step after loading data. The goal is to understand the data's structure, distributions, relationships, and quality issues before any modelling.

Key findings from EDA

- Class imbalance: 73.5% retained vs 26.5% churned — needed to be addressed during training
- TotalCharges: Loaded as a string due to whitespace values — required cleaning with `pd.to_numeric()`
- Contract type: Month-to-month customers churn at ~43% vs 11% for one-year and 3% for two-year contracts
- Tenure: Strongly negatively correlated with churn — newer customers churn far more
- MonthlyCharges: Churned customers have higher average monthly charges (~\$74 vs \$61)
- Fiber optic customers churn at ~42% vs DSL at ~19% — a significant signal

Tools used

- pandas for data loading, cleaning, and aggregation
- matplotlib and seaborn for visualizations
- Correlation heatmap to identify multicollinearity between numerical features

Q: Why did you use AUC-ROC instead of accuracy as your metric?

Accuracy is misleading on imbalanced datasets. If I always predicted 'not churn', I'd get 73.5% accuracy without learning anything useful. AUC-ROC measures the model's ability to rank a random churner above a random non-churner across all thresholds, which is a much better measure of discriminative power. I also used Precision-Recall AUC because it focuses specifically on the minority class (churners), which is what we care most about.

3. Feature Engineering

Feature engineering is the process of creating new input variables from raw data that better capture the underlying patterns. This is often where the biggest performance gains come from — more so than model selection.

Feature	Formula / Logic	Why It Matters
tenure_band	Cut tenure into: new (0-12m), developing, established, loyal (48-72m)	Loyalty segments churn at very different rates
charges_per_month	TotalCharges / (tenure + 1)	Detects sudden billing spikes vs gradual growth
service_count	Count of active services (phone, internet, etc.)	More services = stickier customer
is_monthly	1 if Contract = Month-to-month, else 0	Strongest single predictor of churn
no_support	1 if no OnlineSecurity AND no TechSupport	Proxy for dissatisfaction / disengagement
payment_friction	1 if PaperlessBilling AND Electronic check	Correlated with churn in EDA

Encoding strategy

- Binary columns (Yes/No): Converted to 0/1 integers
- Multi-class columns: One-hot encoded with `pd.get_dummies(drop_first=True)` to avoid multicollinearity
- Numerical features: Standardized with `StandardScaler` (mean=0, std=1)

Q: Why did you drop gender from the features?

Gender had very low predictive power in the EDA — the churn rate was nearly identical across male and female customers. More importantly, using gender as a predictive feature raises ethical concerns around fairness and bias in ML systems. Removing it had no measurable impact on model performance and is the right decision from a responsible AI perspective.

4. Handling Class Imbalance

With 26.5% churn rate, the dataset is imbalanced. If left unaddressed, the model will be biased toward predicting 'not churn' to maximize accuracy, missing the churners we actually care about. There are three main strategies to handle this:

SMOTE — Synthetic Minority Oversampling Technique (chosen approach)

SMOTE generates synthetic training examples for the minority class (churners) by interpolating between existing minority samples in feature space. It does NOT simply duplicate rows — it creates new, realistic data points. Applied only to the training set, never the test set.

```
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train_bal, y_train_bal = smote.fit_resample(X_train, y_train)
# Result: 50/50 balanced training set
```

Other approaches considered

- Class weights: Pass `class_weight='balanced'` to sklearn models — penalizes misclassifying minority class more heavily
- Undersampling: Randomly remove majority class samples — simpler but loses information
- Threshold tuning: Lower the decision threshold from 0.5 to 0.4 to increase recall on churners

Q: Why did you apply SMOTE only on the training set and not the full dataset?

This is a critical point. Applying SMOTE before splitting would cause data leakage — synthetic samples derived from test set points would appear in training, making the model appear better than it actually is. The test set must always represent real, unseen data. SMOTE is fit and applied only on `X_train`, after the train-test split.

5. Model Training & Selection

Five models were trained and compared using stratified 5-fold cross-validation on the balanced training set. Stratified k-fold ensures each fold has the same class distribution as the full dataset, giving more reliable performance estimates.

Why XGBoost was chosen

- Best AUC-ROC (0.874) and PR-AUC (0.702) on held-out test set
- Gradient boosting builds trees sequentially, each correcting errors from the previous
- Built-in regularization (L1/L2) prevents overfitting
- Handles mixed feature types well without extensive preprocessing
- Native support for feature importance and SHAP explainability

How XGBoost works (explain in interview)

XGBoost is an ensemble of decision trees built using gradient boosting. It starts with a simple prediction (the mean), then trains a tree to predict the residual errors. Each subsequent tree corrects the mistakes of all previous trees. The learning rate controls how much each tree contributes. Regularization terms (L1 and L2) are added directly to the loss function to control tree complexity and prevent overfitting.

Key hyperparameters

- `n_estimators=300`: Number of trees in the ensemble
- `learning_rate=0.05`: How much each tree contributes (lower = more robust, slower)
- `max_depth=6`: Maximum depth of each tree (controls complexity)

- `eval_metric='logloss'`: Loss function for binary classification

Q: Why did you use cross-validation instead of a single train/test split for model selection?

A single train/test split gives a noisy estimate of model performance — it depends heavily on which samples happened to end up in the test set. Stratified 5-fold cross-validation trains and evaluates the model 5 times on different splits, giving a more reliable and less variable estimate. I used cross-validation for model selection, then evaluated the final chosen model on a completely held-out test set that was never used during selection.

6. Evaluation Metrics — Explained

Choosing the right metric is as important as choosing the right model. For churn prediction with imbalanced classes, the following metrics are most meaningful:

Metric	What it measures	Our score	Interpretation
AUC-ROC	Ability to rank churners above non-churners at all thresholds	0.874	Model is right 87.4% of the time when ranking a random pair
PR-AUC	Precision-Recall tradeoff specifically for the minority class	0.702	Strong performance on churners specifically
F1 Score	Harmonic mean of precision and recall at chosen threshold	0.648	Good balance at threshold=0.40
Accuracy	Overall correct predictions (NOT used for model selection)	0.829	Misleading due to class imbalance

Threshold tuning

The default decision threshold is 0.5 — predict churn if probability > 0.5. But this is not always optimal. In churn prediction, missing a churner (false negative) is more costly than incorrectly flagging a loyal customer (false positive). By lowering the threshold to 0.40, we increase recall (catch more churners) at the cost of slightly lower precision (some false alarms). The optimal threshold depends on the business's cost structure.

Q: What is the difference between precision and recall, and which matters more for churn?

Precision is: of all customers we predicted would churn, how many actually did? Recall is: of all customers who actually churned, how many did we catch? For churn prediction, recall matters more — missing a churner means losing that customer forever. A false positive (wrongly flagging a loyal customer) just means wasting a small retention offer, which is a much cheaper mistake. So we tune the threshold to favour recall.

7. SHAP Explainability

SHAP (SHapley Additive exPlanations) is a method for explaining individual predictions from any machine learning model. It is based on Shapley values from cooperative game theory — each feature is given a 'credit' for its contribution to a specific prediction.

How SHAP works

For a given prediction, SHAP asks: 'how much did each feature push the prediction above or below the baseline (average) prediction?' A positive SHAP value means the feature increased the predicted churn probability. A negative value means it decreased it.

Three SHAP outputs in this project

- Global bar chart: Mean absolute SHAP value per feature — shows which features matter most overall
- Beeswarm plot: Each dot is one customer. Shows both magnitude and direction of each feature's effect
- Force plot: Single customer explanation — 'this customer is high-risk because of X, Y, Z'

Top features by SHAP importance

- tenure: Low tenure strongly increases churn probability (new customers are most at risk)
- Contract_Month-to-month: Being on a monthly contract is the strongest positive churn driver
- MonthlyCharges: Higher charges increase churn probability
- is_monthly: Engineered flag — high SHAP value confirms feature engineering was effective
- InternetService_Fiber optic: Fiber customers churn more than DSL customers

Q: Why did you use SHAP instead of just feature importance from XGBoost?

XGBoost's built-in feature importance measures how often a feature is used in tree splits, but it doesn't tell you the direction or magnitude of the effect for individual predictions. SHAP gives consistent, theoretically grounded explanations at both the global level (which features matter most overall) and the individual level (why did THIS customer get a high churn score). This is critical for business use — a data scientist needs to explain to a stakeholder not just 'the model thinks this customer will churn', but 'because they are on a monthly contract, have high charges, and no tech support.'

8. MLflow Experiment Tracking

MLflow is an open-source platform for managing the ML lifecycle, including experiment tracking, model versioning, and deployment. In this project, it was used to log and compare all five model experiments.

What was logged for each experiment

- Parameters: All model hyperparameters (n_estimators, learning_rate, max_depth, etc.)
- Metrics: CV AUC, Test AUC, PR-AUC, F1 score
- Model artifact: The trained model saved and versioned in MLflow's model registry
- Run name: Human-readable name for each experiment (e.g. 'XGBoost', 'LightGBM')

Why MLflow matters for a Data Scientist role

In production ML teams, data scientists run dozens or hundreds of experiments. Without tracking, it is impossible to reproduce results or know which model is deployed. MLflow solves this by creating a searchable, versioned record of every experiment. It also provides a model registry for staging models before production deployment, and integrates with deployment tools like Docker and Kubernetes.

Q: Have you used any other experiment tracking tools?

MLflow was used in this project. Other popular alternatives include Weights & Biases (wandb) which is widely used in deep learning for logging training curves and media, and Neptune.ai. The concepts are the same across tools — logging parameters, metrics, and artifacts for each run so experiments are reproducible and comparable.

9. Model Deployment with FastAPI

After training, the model was deployed as a REST API using FastAPI — a modern, fast Python web framework for building APIs. This enables any application (CRM system, mobile app, data pipeline) to request churn predictions in real time.

API endpoints

- GET /health — returns model status and type
- POST /predict — accepts a single customer record, returns churn probability + risk tier + action
- POST /predict/batch — accepts a list of customers, returns predictions for all

Sample API response

```
{
  "churn_probability": 0.7823,
  "will_churn": true,
  "risk_tier": "HIGH",
  "recommended_action": "Immediate outreach – offer retention discount"
}
```

Key technical choices

- Pydantic models: Used for input validation — if a required field is missing or wrong type, the API returns a clear error automatically
- joblib: Used to serialize and deserialize the trained model and scaler
- Preprocessing pipeline: The API replicates the same feature engineering steps used in training to avoid training-serving skew

Q: What is training-serving skew and how did you prevent it?

Training-serving skew happens when the data preprocessing applied during training is different from what is applied at prediction time. For example, if the scaler was fit on training data but prediction inputs are not scaled, the model receives inputs in a completely different range than it was trained on, and predictions will be wrong. I prevented this by saving the fitted StandardScaler with joblib alongside the model, and loading both in the API to apply identical transformations to every incoming request.

10. Streamlit Business Dashboard

The Streamlit dashboard translates model output into a tool that business stakeholders can actually use — without writing any code. It provides interactive exploration of churn risk across the customer base.

Dashboard components

- KPI cards: AUC-ROC, F1 score, total customers flagged, high-risk count

- Threshold slider: Lets the user adjust the risk threshold and see metrics update in real time
- Score distribution: Histogram of churn probabilities split by actual churn label
- Risk tier pie chart: Breakdown of customers into HIGH / MEDIUM / LOW tiers
- SHAP feature importance bar chart: Top 15 features driving churn
- High-risk customer table: Colour-coded table of top N riskiest customers
- Individual explorer: Enter any customer index to see their score and all features

Q: Why did you build a dashboard when the model already has an API?

The API is for technical consumers — other systems and pipelines. The dashboard is for business consumers — a retention manager who needs to decide which customers to call today. A good data science project delivers value to both technical and non-technical stakeholders. The SHAP charts in the dashboard also serve as a trust-building tool — when a manager can see exactly why the model flagged a customer, they are far more likely to act on the recommendation than if they just receive a number.

11. Business Impact & Intervention Strategy

A model that isn't connected to a business decision has no value. The final deliverable is a three-tier intervention strategy that tells the retention team exactly what to do with each customer segment.

Risk Tier	Score Range	Est. Customers	Recommended Action	Cost
HIGH	$\geq 70\%$	~180	Immediate call + retention offer	High
MEDIUM	40-70%	~520	Automated email campaign	Low
LOW	$< 40\%$	~706	No action needed	Zero

Key business insight: Targeting only HIGH + MEDIUM risk customers (~700 out of 7,043) captures approximately 65% of all actual churners — while spending less than 10% of the budget that a blanket 'contact everyone' strategy would require.

Q: How would you measure whether the model is actually working in production?

I would track three things monthly: (1) Actual churn rate of customers who were flagged HIGH risk but received no intervention — this is the model's true positive rate in the wild. (2) Retention rate of HIGH risk customers who did receive an offer — this measures the business value of acting on predictions. (3) PSI (Population Stability Index) on input features — if the customer base changes significantly, the model may need retraining. I would also run an A/B test where one group receives interventions based on the model and a control group receives no intervention, to measure the causal impact of the model.

12. Your Complete 2-Minute Interview Answer

Memorize this structure. Practice saying it out loud until it feels natural. Aim for 90-120 seconds.

I built an end-to-end customer churn prediction system for a telecom company using the IBM Telco dataset with 7,000 customers. The business problem was straightforward: acquiring a new customer

costs 5 to 7 times more than retaining one, so identifying at-risk customers early is extremely valuable. I started with EDA, where I found that month-to-month contract customers churn at 43% — much higher than annual contracts. I then engineered six new features including contract risk flags, service bundle counts, and payment friction indicators. The dataset had a 26.5% churn rate, so I used SMOTE to balance the training set before comparing five models. XGBoost performed best with 0.874 AUC-ROC. I used SHAP to make predictions explainable — both globally and for individual customers — and MLflow to track all experiments. Finally, I deployed the model as a FastAPI endpoint and built a Streamlit dashboard for business stakeholders. The output is a three-tier risk strategy: high-risk customers get immediate outreach, medium-risk get automated campaigns, and low-risk require no action. Targeting just the top two tiers captures 65% of all churners at a fraction of the cost of contacting everyone.

13. Common Follow-Up Questions & Answers

Q: What would you do to improve this model?

Several things: (1) Hyperparameter tuning with Optuna or GridSearchCV to systematically find better XGBoost parameters. (2) Try neural network approaches like TabNet which handles tabular data well. (3) Add customer support ticket data and usage logs if available — behavioral signals often improve churn models significantly. (4) Implement a feedback loop where actual churn outcomes are fed back to retrain the model monthly.

Q: How would you handle a situation where the model's performance degrades over time?

This is called model drift, and it happens when the real world changes but the model doesn't. I would set up automated monitoring using tools like Evidently AI to track data drift (PSI on input features) and prediction drift (shift in score distribution). If PSI exceeds 0.2 on key features or AUC drops more than 3-5 points on a holdout window, I would trigger a retraining pipeline on the latest data.

Q: What is overfitting and how did you prevent it?

Overfitting is when a model learns the training data too well — including noise — and performs poorly on new data. I prevented it through: (1) Cross-validation instead of a single train/test split for model selection. (2) XGBoost's built-in L1/L2 regularization. (3) `max_depth=6` to limit tree complexity. (4) Comparing CV AUC vs test AUC — if they were very different, the model was overfitting.

Q: Why did you choose FastAPI over Flask?

FastAPI has several advantages for ML serving: automatic input validation via Pydantic models, automatic API documentation at `/docs`, async support for high throughput, and it is significantly faster than Flask. For a production ML endpoint that might receive many concurrent requests, FastAPI is the better choice. Flask is simpler and has a larger ecosystem, so it is still a valid choice for simpler use cases.

Q: What is the difference between bagging and boosting?

Bagging (e.g. Random Forest) trains multiple models in parallel on random subsets of the data and averages their predictions. It reduces variance. Boosting (e.g. XGBoost, LightGBM) trains models sequentially, where each model corrects the errors of the previous one. It reduces both bias and variance but is more prone to overfitting without regularization. Boosting generally achieves higher accuracy on tabular data, which is why XGBoost is the most popular model for structured data

competitions.

Q: How would you deploy this to serve 1 million customers daily?

For batch scoring at scale: run the model as a nightly batch job using Apache Spark or a distributed pandas alternative like Dask, write scores to a database table, and let the CRM query it. For real-time serving at scale: containerize the FastAPI app with Docker, deploy behind a load balancer on Kubernetes, and cache frequent predictions. Model serving latency should be under 100ms, which XGBoost easily achieves.

14. Key Terms to Know

AUC-ROC: Area Under the ROC Curve. Measures discriminative ability across all thresholds. 0.5 = random, 1.0 = perfect.

SMOTE: Synthetic Minority Oversampling Technique. Generates synthetic samples for minority class by interpolating between existing samples.

SHAP: SHapley Additive exPlanations. Assigns each feature a contribution value for a specific prediction, grounded in game theory.

Gradient Boosting: Ensemble method that builds trees sequentially, each correcting the residual errors of the previous trees.

Cross-validation: Technique to estimate model performance by training and evaluating on multiple non-overlapping data splits.

Data leakage: When information from outside the training dataset is used to create the model, leading to overly optimistic performance estimates.

Precision: Of all predicted positives, how many are actually positive. $TP / (TP + FP)$.

Recall: Of all actual positives, how many did we correctly predict. $TP / (TP + FN)$.

F1 Score: Harmonic mean of precision and recall. Useful when both matter and classes are imbalanced.

Feature importance: Measure of how much each feature contributes to the model's predictions, either globally or locally.

Model drift: Degradation of model performance over time as real-world data distribution changes from training data.

PSI: Population Stability Index. Measures how much a feature's distribution has shifted between two time periods.

REST API: Representational State Transfer. Architectural style for web APIs using HTTP methods (GET, POST, PUT, DELETE).

Pydantic: Python library for data validation using Python type hints. Used in FastAPI for request/response validation.

joblib: Python library for serializing Python objects (like trained ML models) to disk and loading them back.

**You built this. You understand it.
You can explain it.**

Good luck, Kashvi. You've got this. ■